

Assignment 3: End-to-End ML Versioning

Part D: DVC Pipeline (dvc.yaml + params.yaml)

Student: Munam

Repository: github.com/Munam32/MLOps

Tags: v1 (dense_units 128), v2 (dense_units 256)

Four terms used throughout

Stage	One step of the pipeline: a command, the files it needs (<i>deps</i>) and the files it makes (<i>outs</i>).
params.yaml	The one file holding every hyperparameter. Scripts read it instead of hard-coding values.
dvc repro	Runs the pipeline, but only the stages whose deps or params changed since the last run.
dvc.lock	Written by <code>dvc repro</code> . Records the exact hash of every dep, param value and output, so DVC can tell what changed.

D1 params.yaml

Goal: keep all hyperparameters in one file, one section per stage.

`preprocess` holds the validation split size and the random seed. `train` holds the five model settings.

`preprocess.py` and `train.py` each load only their own section with `yaml.safe_load()`. `prepare.py` and `evaluate.py` need no settings.

params.yaml, and where the scripts read it

```
$ cat params.yaml
preprocess:
  val_size: 0.1
  seed: 42

train:
  dense_units: 128
  dropout_rate: 0.2
  learning_rate: 0.001
  epochs: 20
  batch_size: 32

$ grep -n 'safe_load' src/*.py
src/preprocess.py:16:     params = yaml.safe_load(f)["preprocess"]
src/train.py:17:     params = yaml.safe_load(f)["train"]
```

D2 dvc.yaml

Goal: describe the four stages and how they connect.

dvc.yaml

```
$ cat dvc.yaml
```

```
stages:
```

```
  prepare:
```

```
    cmd: python src/prepare.py
```

```
    deps:
```

```
      - src/prepare.py
```

```
    outs:
```

```
      - data/raw
```

```
  preprocess:
```

```
    cmd: python src/preprocess.py
```

```
    deps:
```

```
      - src/preprocess.py
```

```
      - data/raw
```

```
    params:
```

```
      - preprocess.val_size
```

```
      - preprocess.seed
```

```
    outs:
```

```
      - data/processed
```

```
  train:
```

```
    cmd: python src/train.py
```

```
    deps:
```

```
      - src/train.py
```

```
      - data/processed
```

```
    params:
```

```
      - train.dense_units
```

```
      - train.dropout_rate
```

```
      - train.learning_rate
```

```
      - train.epochs
```

```
      - train.batch_size
```

```
    outs:
```

```
      - models
```

```
  evaluate:
```

```
    cmd: python src/evaluate.py
```

```
    deps:
```

```
      - src/evaluate.py
```

```
      - models
```

```
      - data/processed
```

```
    metrics:
```

```
      - metrics.json:
```

```
        cache: false
```

```
    plots:
```

```
      - reports/confusion_matrix.png:
```

```
        cache: false
```

Stage order DVC works out from dvc.yaml

```
$ dvc dag
```

```
+-----+
| prepare |
+-----+
  *
  *
  *
+-----+
| preprocess |
+-----+
**      **
**      *
*      **
+-----+
| train |
+-----+
**      **
**      *
  *
+-----+
| evaluate |
+-----+
```

STAGE	NEEDS	MAKES
prepare	its script	data/raw
preprocess	data/raw + 2 params	data/processed
train	data/processed + 5 params	models
evaluate	models, data/processed	metrics.json, plot

DVC works out the order from these links: one stage's output is the next stage's input. `metrics.json` is listed under `metrics:` rather than `outs:`, so `dvc metrics diff` can compare it between versions. `cache: false` keeps it and the plot in Git instead of the DVC cache.

D3 Run the whole pipeline

Goal: run all four stages with one command and get a `dvc.lock`.

To show every stage executing, this log comes from a fresh checkout of tag `v1` with an empty DVC cache, so nothing could be skipped. The four stages run in dependency order. After each stage DVC writes its hashes to `dvc.lock` ("Updating lock file"). Training prints one line per epoch, shortened here.

dvc repro on a clean checkout of v1

```
$ dvc repro
Running stage 'prepare':
> python src/prepare.py
Saved raw train (60000 samples) and test (10000 samples) data to <clean checkout of v1>/data/raw
Running stage 'preprocess':
> python src/preprocess.py
Saved processed train (54000), val (6000), and test (10000) data to <clean checkout of v1>/data/processed
Running stage 'train':
> python src/train.py
Epoch 1/20 1688/1688 - accuracy: 0.8087 - val_accuracy: 0.8513
Epoch 2/20 1688/1688 - accuracy: 0.8531 - val_accuracy: 0.8707
... epochs 3-19 omitted
Epoch 20/20 1688/1688 - accuracy: 0.9106 - val_accuracy: 0.8973
Saved trained model and history to <clean checkout of v1>/models
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Test loss: 0.3484, test accuracy: 0.8874
Saved metrics.json to <clean checkout of v1> and confusion_matrix.png to <clean checkout of v1>/reports
Updating lock file 'dvc.lock'
Use `dvc push` to send your updates to remote storage.
$ ls dvc.lock
dvc.lock
```

In short: one `dvc repro` rebuilt everything from scratch and reached **88.7% test accuracy**. The committed v1 run reached 88.82%. The small difference is normal: training starts from random weights each time.

D4 Change one hyperparameter and re-run

Goal: see which stages DVC re-runs after a change.

I doubled `dense_units` from 128 to 256. Before running anything, `dvc status` already reports that only `train` is affected.

Change one hyperparameter

```
$ git diff params.yaml
diff --git a/params.yaml b/params.yaml
index 1174722..4c8b3c0 100644
--- a/params.yaml
+++ b/params.yaml
@@ -3,7 +3,7 @@ preprocess:
     seed: 42

train:
-   dense_units: 128
+   dense_units: 256
     dropout_rate: 0.2
     learning_rate: 0.001
     epochs: 20

$ dvc status
train:
    changed deps:
        params.yaml:
            modified:          train.dense_units
```

dvc repro after the change

```
$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train.py
Epoch 1/20 1688/1688 - accuracy: 0.8160 - val_accuracy: 0.8667
Epoch 2/20 1688/1688 - accuracy: 0.8574 - val_accuracy: 0.8722
... epochs 3-19 omitted
Epoch 20/20 1688/1688 - accuracy: 0.9158 - val_accuracy: 0.9033
Saved trained model and history to ~/Desktop/MLOps/models
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Test loss: 0.3420, test accuracy: 0.8877
Saved metrics.json to ~/Desktop/MLOps and confusion_matrix.png to ~/Desktop/MLOps/reports
Updating lock file 'dvc.lock'
Use `dvc push` to send your updates to remote storage.
```

STAGE	RESULT	WHY
prepare	Skipped	Its script and outputs are unchanged, and it uses no params.
preprocess	Skipped	It depends only on <code>preprocess.*</code> params, and those didn't change.
train	Re-ran	It depends on <code>train.dense_units</code> , which changed.
evaluate	Re-ran	Its input, <code>models</code> , was just rebuilt by <code>train</code> .

In short: DVC compares every stage's current deps and params with the hashes in `dvc.lock`. A stage runs again only if something it depends on changed, directly or because an earlier stage produced new output. The two data stages were skipped, which saved re-downloading and re-normalising the dataset.

D5 Commit, tag v2, push

Goal: save v2 in Git and send its data and model to Google Drive alongside v1.

Commit and tag

```
$ git add params.yaml dvc.yaml dvc.lock metrics.json reports/confusion_matrix.png

$ git status --short
M dvc.lock
M metrics.json
M params.yaml
M reports/confusion_matrix.png

$ git commit -q -m 'exp: increase dense_units from 128 to 256' -m 'Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>' && git log --oneline -1
c63beae exp: increase dense_units from 128 to 256

$ git tag v2

$ git log --oneline --decorate -3
c63beae (HEAD -> dev, tag: v2) exp: increase dense_units from 128 to 256
7c4fcac (tag: v1, origin/dev) feat: add DVC pipeline and replace dvc pointers
6065a4a chore: delete dvc.yaml
```

dvc push: v2 outputs sent to the Google Drive remote

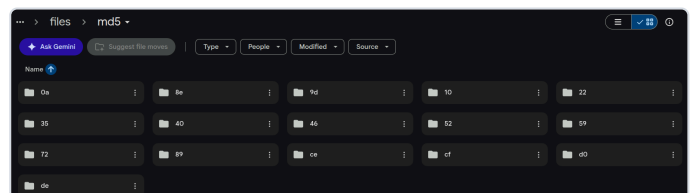
```
$ git checkout dev
Switched to branch 'dev'

$ dvc push
Collecting
Pushing
3 files pushed
```

v1 vs v2, straight from DVC

```
$ dvc params diff v1 v2
Path      Param          v1    v2
params.yaml train.dense_units 128   256

$ dvc metrics diff v1 v2
Path      Metric          v1      v2      Change
metrics.json test_accuracy 0.8882  0.8877  -0.0005
metrics.json test_loss  0.3448  0.3420  -0.0028
```



The Google Drive remote after the push. DVC stores each file under `files/md5/`, in a folder named after the first two characters of its hash, so v1 and v2 files sit side by side and nothing is overwritten.

VERSION	DENSE_UNITS	TEST ACCURACY	TEST LOSS	TRAINABLE PARAMS
v1	128	88.82%	0.3448	101,770
v2	256	88.77%	0.3420	203,530
Change		-0.05 pts	-0.0028	×2.0

In short: doubling the hidden layer doubled the model size but didn't improve accuracy. Test loss is marginally lower, and accuracy is within run-to-run noise. The 128-unit model is already large enough for this dataset, so v1 remains the better choice. Because both versions are tagged, `git checkout v1` && `dvc checkout` restores the exact v1 model and data at any time.